

Informe de la pràctica 7 Busos de comunicació III

Introducció

Aquesta setena pràctica culmina l'estudi dels busos de comunicació en sistemes encastats introduint el protocol **I2S (Inter-IC Sound)**. Desenvolupat originalment per Philips Semiconductors l'any 1986, aquest bus sèrie és l'estàndard de la indústria per connectar dispositius d'àudio digital. A través d'aquesta pràctica, explorarem com un microcontrolador pot llegir arxius musicals complexos, processar-los i convertir-los en un flux ininterromput de dades per reproduir so d'alta qualitat.

1.1. Objectius

La finalitat d'aquest projecte és entendre les particularitats de la transmissió d'àudio digital en temps real i aprendre a combinar diferents busos de maquinari. Els objectius específics són:

- **Assimilar l'arquitectura del bus I2S:** Comprendre la necessitat de les línies de rellotge específiques i com es diferencien els canals estèreo en la transmissió de dades digitals.
- **Conversió Digital-Analògica (DAC):** Estudiar la integració i configuració d'un descodificador i amplificador extern (mòdul MAX98357A) per transformar els bits d'àudio en senyals elèctrics capaços de moure un altaveu.
- **Descodificació des de memòria interna:** Aconseguir reproduir un arxiu d'àudio comprimit (en format AAC) directament des de la memòria flaix del programa de l'ESP32, fent ús de la directiva **PROGMEM** per optimitzar l'espai.
- **Multitasca i fusió de busos (SPI + I2S):** Implementar un sistema capaç de llegir un arxiu WAVE (.wav) des d'un sistema de fitxers extern (targeta SD via SPI) i retransmetre'l a l'instant cap a l'amplificador mitjançant el bus I2S, tot gestionant els esdeveniments asíncrons i les metadades.

1.2. Material Utilitzat

Per desenvolupar aquesta pràctica dedicada a l'àudio, s'ha fet servir el següent maquinari i programari específic:

Maquinari (Hardware):

- **Microcontrolador:** Placa de desenvolupament ESP32-S3. L'ESP32 és ideal per aquesta pràctica perquè té suport nadiu de maquinari per a l'I2S i prou potència per descodificar àudio, a diferència de plaques clàssiques com l'Arduino Uno.
- **Mòdul I2S:** Placa de connexió d'àudio MAX98357A, que actua de descodificador DAC i amplificador de classe D.

- **Altaveu:** Un altaveu estàndard (ex. 4 o 8 ohms) connectat a la sortida de l'amplificador.
- **Emmagatzematge:** Mòdul adaptador de targetes micro SD amb una targeta de memòria (per al segon exercici).
- Placa de proves (*proto board*) i cables de connexió Dupont.

Programari (Software):

- Entorn de desenvolupament Visual Studio Code amb l'extensió PlatformIO.
- Llibreria **ESP8266Audio** (de Earle F. Philhower) per a la reproducció d'àudio i manipulació del format AAC des de la memòria PROGMEM.
- Llibreria **ESP32-audioI2S** (de l'ecosistema ESPHome) per a la gestió avançada de fluxos d'àudio i lectura integrada d'arxius WAVE des de la SD.
- Llibreries del sistema: **SPI.h**, **SD.h** i **FS.h** per al control del bus SPI i el sistema de fitxers.

1.3. Conceptes Claus: Per què utilitzar l'I2S?

Una de les preguntes fonamentals que resol aquesta pràctica és per què no es connecta l'altaveu directament al microcontrolador. L'ESP32 té un convertidor Digital-Analògic (DAC) intern, però només té una resolució de 8 bits. Si intentéssim reproduir àudio professional (que habitualment està mostrejat a 44,1 kHz i té una profunditat de 16 bits), perdríem moltíssima qualitat al truncar les dades.

El protocol de comunicació I2S soluciona això traient el so fora del xip en format 100% digital i sense pèrdues, utilitzant només tres cables sincronitzats:

- **SCK / BCLK (Serial Clock / Bit Clock):** El rellotge que sincronitza els dispositius, generat pel mestre (l'ESP32). La seva velocitat depèn de la qualitat de l'arxiu ($\text{Frec. de mostreig} \times \text{Bits per canal} \times \text{N}^\circ \text{ de canals}$).
- **WS / LRC (Word Select / Left-Right Clock):** És la línia que diferencia entre el canal esquerre (0) i el dret (1), indicant a l'amplificador quin altaveu ha de reproduir el so.
- **SD / DIN (Serial Data):** El cable per on viatja la càrrega útil (els bits de la cançó), on sempre s'envia primer el bit més significatiu (MSB).

Desenvolupament i Implementació: Exercici Pràctic 1 (Reproducció des de memòria interna)

L'objectiu d'aquest primer exercici és comprendre el funcionament del protocol de comunicació I2S (Inter-IC Sound) transmetent un arxiu d'àudio digital des del microcontrolador ESP32 cap a un descodificador extern (el mòdul amplificador MAX98357A) per reproduir-lo per un altaveu. En lloc d'utilitzar una targeta SD d'inici, l'arxiu de so s'emmagatzema directament dins la memòria flash interna de la placa.

1. Preparació de l'Arxiu d'Àudio (**sampleaac.h**)

Un microcontrolador no pot llegir directament un arxiu d'àudio convencional (.mp3, .wav, .aac) com si fos un ordinador si no disposa d'un sistema de fitxers muntat. Per solucionar-ho i poder-lo integrar dins del propi codi font, s'ha dut a terme un procés de tractament previ:

1. S'ha descarregat una cançó des de la plataforma YouTube.
2. Com que l'espai de memòria de l'ESP32 és limitat, l'àudio s'ha retallat a un fragment de curta durada.
3. El fragment s'ha convertit a un format codificat eficient (AAC) i, mitjançant una eina de conversió a codi C, s'ha transformat en un vector (array) de valors hexadecimals d'una mida de 876.220 bytes.
4. Aquest array s'ha desat a l'arxiu auxiliar `sampleaac.h` sota la declaració `const unsigned char sampleaac[] PROGMEM = { 0xff, 0xf1... }.`

L'ús de la directiva **PROGMEM** és fonamental en aquest pas. Si intentéssim carregar un array de gairebé 900 KB a la memòria RAM convencional de l'ESP32 (que en té uns 512 KB), la placa col·lapsaria immediatament. L'etiqueta **PROGMEM** obliga el compilador a guardar aquestes dades a la memòria flaix del programa, que és molt més gran i permet llegir-les progressivament sense saturar la RAM.

2. Configuració de l'Entorn (`platformio.ini`)

Per poder extreure l'àudio de l'array, descodificar el format AAC i enviar-lo pel maquinari I2S, s'ha afegit una dependència imprescindible al fitxer de configuració:

- **lib_deps = earlephilhower/ESP8266Audio @ ^1.9.7**: Aquesta llibreria és una de les més potents per al tractament d'àudio en microcontroladors de la família ESP. Ens abstrau de la complexitat matemàtica de descodificar els formats d'àudio i s'encarrega de generar automàticament els senyals de rellotge (BCLK) i selecció de canal (LRC) necessaris per a l'I2S.

3. Codi Principal: Descodificació i Reproducció I2S (`main.cpp`)

L'arquitectura del codi es basa en l'ús de tres objectes principals de la llibreria **ESP8266Audio** que actuen com una canonada de dades (*pipeline*): la font, el descodificador i la sortida.

Inicialització i Configuració (`setup`):

- **in = new AudioFileSourcePROGMEM(sampleaac, sizeof(sampleaac));**: Es crea la font d'entrada. Aquesta línia indica al programa que les dades en brut s'han de llegir des de la memòria flaix (PROGMEM), passant-li el nom de l'array i la seva mida total.
- **aac = new AudioGeneratorAAC();**: Es crea el motor que s'encarregarà de descodificar l'algorisme de compressió AAC per convertir-lo en dades digitals pures.
- **out = new AudioOutputI2S();**: Es crea l'objecte de sortida que governarà els pins físics del bus I2S de l'ESP32.

- **out->SetGain(0.125);**: S'ajusta el volum al 12,5%. Com que l'amplificador MAX98357A té una potència de fins a 3,2W, és vital abaixar el guany digital des del programari per no danyar l'altaveu ni distorsionar excessivament el so.
- **out->SetPinout(4, 5, 6);**: Aquesta instrucció és crucial per adaptar el codi a l'ESP32-S3. Modifica els pins per defecte i assigna:
 - **Pin 4**: BCLK (*Bit Clock* o Rellotge Sèrie).
 - **Pin 5**: LRC o WS (*Word Select* per diferenciar el canal esquerre/dret).
 - **Pin 6**: DIN o SD (*Serial Data*, on viatgen els bits d'àudio).
- **aac->begin(in, out);**: S'inicia la reproducció establint la connexió lògica entre la font (l'array), el descodificador (AAC) i la sortida (l'altaveu).

El Bucle de Reproducció (loop): A diferència de les comandes bàsiques com un **delay()**, l'àudio no es processa tot de cop.

- L'estructura condicional **if (aac->isRunning())** comprova si encara queda cançó per reproduir.
- Mentre la cançó soni, s'executa **aac->loop();** constantment. Aquesta instrucció agafa un petit fragment de l'array, el descodifica i l'envia pel bus I2S. Ha d'estar al bucle principal sense cap bloqueig perquè l'altaveu rebi un flux de dades ininterromput; qualsevol pausa al **loop** causaria talls o espetecs al so.
- Si la cançó s'acaba (**else**), s'executa **aac->stop()** per aturar el motor i s'inicia una pausa d'1 segon (**delay(1000)**) abans de tornar a repetir el missatge final.

Sortida pel Terminal de Control i Altaveu

Durant la pràctica, quan es carrega el programa a l'ESP32, el monitor sèrie es manté inicialment en silenci. Tanmateix, de manera gairebé immediata, a través de l'altaveu connectat al mòdul MAX98357A es pot escoltar la cançó retallada reproduint-se de forma nítida i fluida.

Un cop passats els segons que dura el fragment musical (el temps que triga l'ESP32 a llegir tots els 876.220 bytes de l'array), el motor de l'àudio s'atura. En aquell moment, el terminal de l'ordinador comença a imprimir el missatge de finalització definit al codi:

```
Plaintext
Sound Generator
Sound Generator
Sound Generator
...
```

(Aquest missatge es repeteix indefinidament cada segon, confirmant que el procés de lectura de memòria s'ha completat amb èxit).

Anàlisi del Temps Lliure del Processador (Reproducció I2S des de PROGMEM)

El comportament del processador en aquest exercici es divideix en dues fases completament oposades, marcades per l'estat de la reproducció de l'àudio.

1. Fase de Reproducció (Càrrega Computacional Alta): Mentre la funció `aac->isRunning()` és certa, el programa executa contínuament la instrucció `aac->loop()` dins del bucle principal, sense cap mena de pausa o `delay`. Durant aquest període, el temps lliure de la CPU (cedit a la Tasca Inactiva) és pràcticament nul. Això es deu a dos factors:

- **Descodificació en temps real:** L'àudio està comprimit en format AAC. El processador ha d'estar constantment llegint blocs de memòria flaix (PROGMEM), aplicant algoritmes matemàtics complexos per descomprimir-los i convertint-los en dades digitals pures (PCM).
- **Manteniment del Buffer:** Aquestes dades s'han d'enviar contínuament al bus I2S perquè l'altaveu no pateixi talls. El `loop()` s'assegura que el *buffer* d'àudio estigui sempre ple.

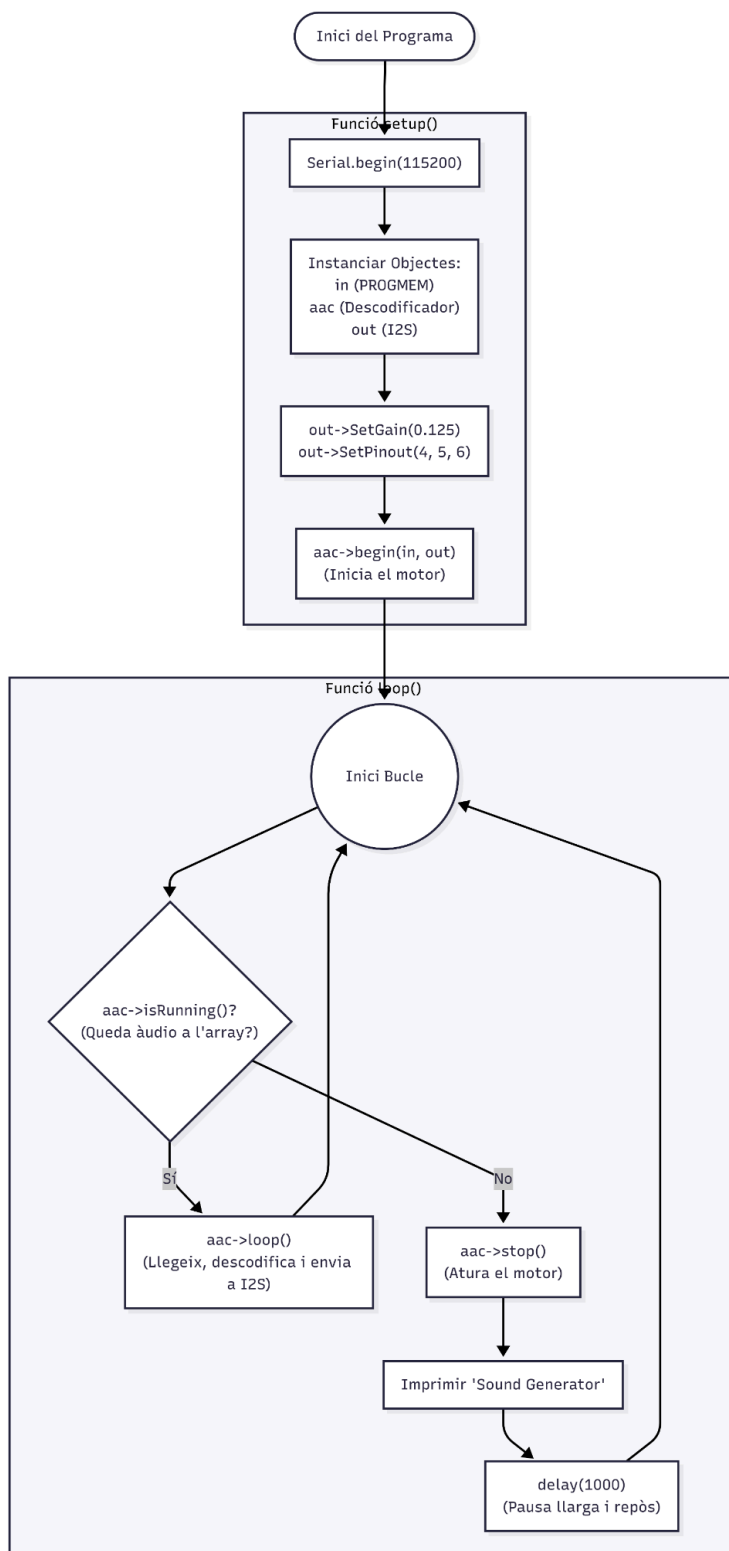
Nota sobre el maquinari (DMA): Tot i que la CPU està molt ocupada descodificant, cal destacar que la transmissió física dels bits cap al mòdul MAX98357A a 1,411 MHz la realitza el controlador **DMA** (*Direct Memory Access*) intern de l'ESP32. Si la CPU hagués de gestionar cada pols de rellotge manualment (com feia l'Arduino Uno), seria incapaç de reproduir àudio d'aquesta qualitat.

2. Fase d'Aturada (Repòs Absolut): Un cop s'han llegit tots els bytes de l'array `sampleaac`, la condició falla i el codi entra a l'estructura `else`. En aquest punt, s'atura el motor d'àudio (`aac->stop()`) i s'executa un `delay(1000)`. A partir d'aquest instant, el processador deixa de fer càlculs matemàtics, el bus I2S es buida i el sistema operatiu FreeRTOS deriva tot el control a la Tasca Inactiva (*Idle Task*) durant aquest segon de pausa. El temps lliure del processador passa sobtadament de gairebé el 0% al 100%.

Conclusió: Aquest codi demostra que el processament d'àudio digital requereix una potència de càlcul sostinguda. A diferència de les lectures esporàdiques de sensors (com a l'I2C o l'SPI), mantenir un flux I2S constant obliga l'ESP32 a treballar a ple rendiment mentre dura la cançó, posant en relleu per què aquests microcontroladors de 32 bits i doble nucli són els indicats per a projectes multimèdia.

Diagrames de flux i temps

Diagrama de flux



La Canonada de Dades

(Setup): El bloc superior mostra la preparació de la memòria, on s'instancien i s'enllacen lògicament la font (l'array emmagatzemat a la memòria flaix PROGMEM), el motor descodificador i la sortida I2S. Destaca especialment el pas de configuració física

`SetPinout(4, 5, 6)` que adapta el programari als pins concrets de l'ESP32-S3.

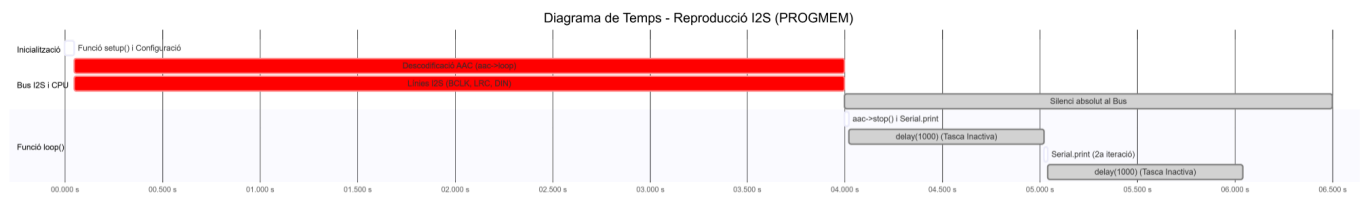
El Cicle d'Alta Càrrega (Rombe

- Opció Sí): La part més important del diagrama és el cicle curt que es forma quan la condició `isRunning()` és certa. S'hi veu clarament que el programa només executa `aac->loop()` i torna a començar de forma frenètica. Aquesta ruta tancada representa el període d'alta càrrega computacional i ocupació del bus I2S que manté la cançó sonant sense interrupcions.

La Fi del Procés (Rombe -

Opció No): Quan l'array s'esgota, el flux canvia dràsticament cap a la dreta. El motor es tanca definitivament amb `stop()` i s'entra en un bucle lent marcat pel bloqueig temporal `delay(1000)`, moment en el qual el bus I2S queda mut i el processador pot finalment descansar.

Diagrama de temps



La Finestra de Reproducció (0 - 4 segons): Immediatament després del `setup()`, el diagrama mostra una franja vermella contínua (crítica). Durant aquest període, l'ESP32 no descansa mai; la CPU executa la instrucció `aac->loop()` ininterrompudament per processar la matemàtica del format AAC i el controlador DMA manté saturades les tres línies del bus I2S (BCLK, LRC i DIN) enviant el flux de bits a l'amplificador MAX98357A.

L'Estat de Repòs (A partir dels 4 segons): Un cop s'esgota l'array de l'àudio guardat a la memòria PROGMEM, l'escenari canvia radicalment. S'envia l'ordre de `stop()`, el bus I2S entra en silenci (cessen els polsos de rellotge) i el programa cau en un cicle repetitiu marcat per un `delay(1000)`. Visualment es demostra com, gràcies a aquest `delay`, la càrrega de treball pràcticament desapareix (zones grises) deixant la CPU en repòs sota el control de la Tasca Inactiva del sistema operatiu (FreeRTOS) i despertant només uns pocs mil·lisegons cada segon per imprimir un missatge de text pel port sèrie.

Desenvolupament i Implementació: Exercici Pràctic 2 (Reproducció d'un arxiu WAVE des de SD)

L'objectiu d'aquest segon exercici és superar la limitació d'espai de la memòria flaix interna de l'ESP32. En lloc d'utilitzar un petit array dins del codi (PROGMEM), muntarem un sistema de fitxers extern mitjançant un mòdul de targeta micro SD, la qual ens permetrà llegir i reproduir arxius d'àudio complets (com un `.wav`) passant la informació en temps real al descodificador MAX98357A.

1. Configuració de l'Entorn (`platformio.ini`)

Per a aquest exercici hem canviat completament la llibreria base per adaptar-nos a les necessitats de llegir un flux de dades extern:

- **`lib_deps = esphome/ESP32-audioI2S @ ^2.0.7`:** Aquesta llibreria (originalment desenvolupada per *schreibfaul1* i mantinguda per l'ecosistema ESPHome) està dissenyada específicament per a l'ESP32. És extremadament potent perquè gestiona automàticament la lectura de sistemes de fitxers (SD, SPIFFS) o fins i tot ràdios per internet (Wi-Fi), descodifica el format de la cançó i la canalitza directament cap al maquinari I2S.
- La velocitat del monitor es manté a **`115200`** bauds per rebre la gran quantitat de metadades que generarà el reproductor.

2. Codi Principal: Fusió dels busos SPI i I2S (**main.cpp**)

L'arquitectura del codi destaca per la separació i configuració de dos protocols de maquinari que funcionaran de forma coordinada.

Definició de Pins (Maquinari Dual): S'han definit dos blocs de pins diferenciats. D'una banda, el bus SPI per a la targeta SD (**CS=10, MOSI=11, SCK=12, MISO=13**) i, de l'altra, el bus I2S per a l'amplificador (**BCLK=4, LRC=5, DOUT=6**).

Inicialització i Seguretat (**setup**):

- **pinMode(SD_CS, OUTPUT); digitalWrite(SD_CS, HIGH);**: Com ja vam aprendre a la pràctica 6, és vital posar el pin *Chip Select* a nivell ALT abans d'iniciar el bus SPI per evitar col·lisions o errors de maquinari amb la targeta SD.
- **SPI.begin(SPI_SCK, SPI_MISO, SPI_MOSI);**: S'inicialitza el bus SPI assignant-li explícitament els pins definits per a l'ESP32-S3.
- **SD.begin(SD_CS, SPI, 4000000);**: Es munta el sistema de fitxers de la SD vinculant-lo a l'objecte SPI. A més, s'estableix una velocitat de 4 MHz per garantir una lectura estable sense desbordaments.
- **audio.setPinout(I2S_BCLK, I2S_LRC, I2S_DOUT);**: Igual que en l'exercici anterior, es configuren els tres cables del bus síncron d'àudio cap a l'amplificador. Es manté el volum controlat a 10 (sobre un màxim de 21) amb **audio.setVolume(10)**.
- **audio.connecttoFS(SD, "/Ensoniq-ZR-76-01-Dope-77.wav");**: Aquesta és la instrucció clau. Fa de pont de connexió. Li estem dient a la llibreria d'àudio que busqui l'arxiu (en aquest cas **.wav**) dins de l'arrel de la targeta de memòria física (**SD**) que acabem de muntar.

El Bucle i els Esdeveniments (**loop** i **Callbacks**):

- L'única instrucció present al bucle principal és **audio.loop();**. Aquesta línia s'ha d'executar milers de vegades per segon; s'encarrega d'extreure un bloc de bytes per l'SPI (des de la SD) i introduir-lo instantàniament per l'I2S cap a l'altaveu.
- La gran novetat d'aquesta llibreria és el bloc final de **funcions de Callback**. Funcions com **audio_info()** o **audio_bitrate()** no són cridades explícitament per nosaltres enlloc del codi principal. Treballen en segon pla: quan la llibreria detecta un esdeveniment important o una dada dins del fitxer **.wav** (metadades), dispara automàticament aquestes funcions per imprimir la informació per pantalla.

Sortida pel Terminal de Control

Quan executem aquest programa, l'ESP32 arrenca muntant el sistema d'arxius i, automàticament, comença a reproduir la cançó. El més interessant, però, és observar el terminal de l'ordinador.

Gràcies a les funcions de *callback*, veurem com el processador destripa el format de l'arxiu d'àudio a l'instant, mostrant-nos informació valuosa de les capçaleres i del flux de dades:

Plaintext

Targeta SD muntada correctament.

info Format is wav

info Data size: 1048576 bytes

info Sample rate: 44100

info Channels: 2

info Bits per sample: 16

bitrate 1411200

(Nota: L'àudio sona mentre es mostren aquests missatges). Un cop finalitzada la cançó de la targeta SD, el terminal ens mostraria el missatge programat `eof_speech` per indicar la fi del fitxer.

Anàlisi del Temps Lliure del Processador (Reproducció I2S des de SD)

En aquest segon exercici, el perfil de rendiment del microcontrolador canvia respecte a l'exercici anterior. Tot i que no hi ha cap pausa explícita al bucle principal, l'ús dels recursos interns de l'ESP32 es distribueix d'una manera diferent:

1. El Processador com a "Pont de Dades" (Alta càrrega de busos): A la funció `loop()`, l'única instrucció que s'executa contínuament és `audio.loop()`. Durant la reproducció, l'ESP32 està sotmès a una forta càrrega d'Entrada/Sortida (I/O) simultània:

- Per una banda, utilitza el bus **SPI** a alta velocitat (4 MHz) per anar extraient blocs de bytes de l'arxiu emmagatzemat a la memòria SD física.
- Gairebé a l'instant, mou aquests bytes cap al controlador DMA intern perquè els transmeti pel bus **I2S** cap a l'amplificador MAX98357A. Com que l'àudio no pot patir interrupcions, el processador gira ràpidament dins d'aquest bucle per garantir que el *buffer* (la memòria d'emmagatzematge temporal) de l'I2S no es quedi mai buit.

2. Descens de la Càrrega Matemàtica (WAV vs AAC): A diferència del primer exercici on calia descomprimir un arxiu AAC (fet que exigia molta potència de càlcul al nucli de la CPU), en aquest cas reproduïm un arxiu format **WAVE** (`.wav`). El format WAV sol contenir àudio PCM (modulació per impulsos codificats) sense comprimir. Això significa que l'ESP32 no ha de fer pràcticament cap càlcul matemàtic complex; simplement actua com una "cinta transportadora", agafant els bits de l'SPI i col·locant-los a l'I2S.

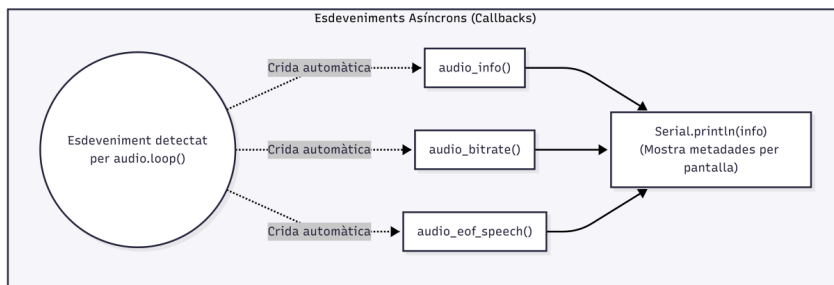
3. Temps Lliure i Gestió Interna: Tot i que al nostre codi no hi ha cap `delay()` per forçar el repòs, el temps lliure no és totalment nul. La potent llibreria `ESP32-audioI2S` gestiona internament el flux de dades. Quan llegeix un bloc de la SD i omple el *buffer* de transmissió I2S, la llibreria cedeix uns breus instants de temps de procés al sistema operatiu (FreeRTOS) mentre espera que el maquinari consumeixi aquestes dades per l'altaveu.

Conclusió: Ens trobem davant d'un programa altament eficient però que manté el processador compromès. La CPU dedica la gran majoria del seu temps a coordinar el trànsit entre els dos busos de maquinari (SPI i I2S). És una demostració pràctica excel·lent de com els microcontroladors moderns poden gestionar fluxos de dades massius i constants en temps real gràcies al treball combinat dels seus perifèrics de maquinari.



Diagrames de flux i temps

Diagrama de flux



Prevenició de Col·lisions (inici del Setup): El diagrama mostra com el primer pas físic és el bloqueig del pin `SD_CS` (posant-lo a `HIGH`) per garantir que el bus SPI quedi net abans d'iniciar les comunicacions generals.

La Barrera de Seguretat (Rombe): Igual que vèiem en exercicis passats, el muntatge de la SD inclou un pas crític. Si la targeta no hi és o està mal connectada, l'ordre `return` expulsa el flux d'execució del `setup()` perquè no intenti reproduir el que no existeix.

El Motor Asíncron: Visualment queda molt clara la importància de la instrucció `audio.loop()`. Mentre el bucle principal només conté aquesta caixa i gira de forma autònoma, s'encarrega en segon pla de disparar (línies discontinúes) totes les funcions auxiliars de *callback* que nodreixen de metadades el port sèrie sense interferir amb el flux de l'àudio.

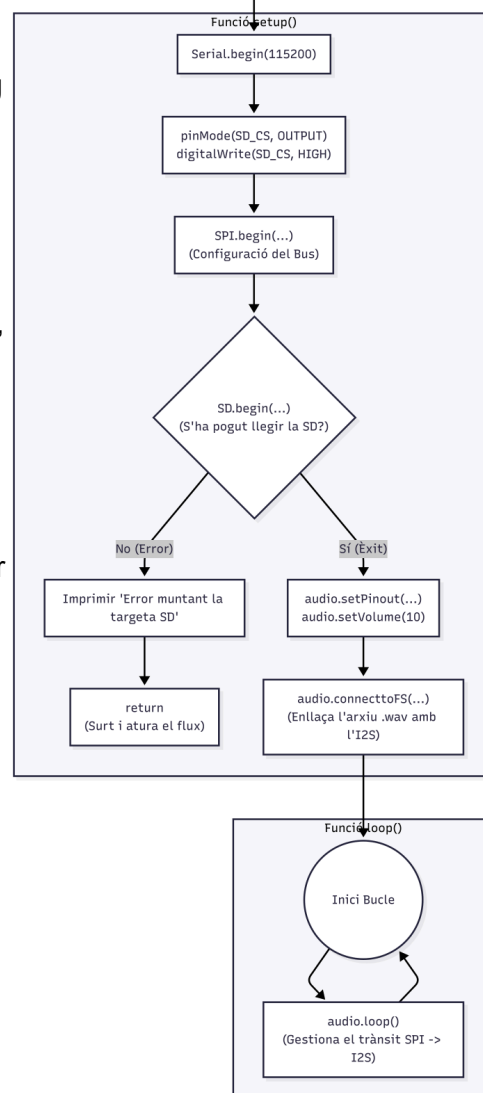
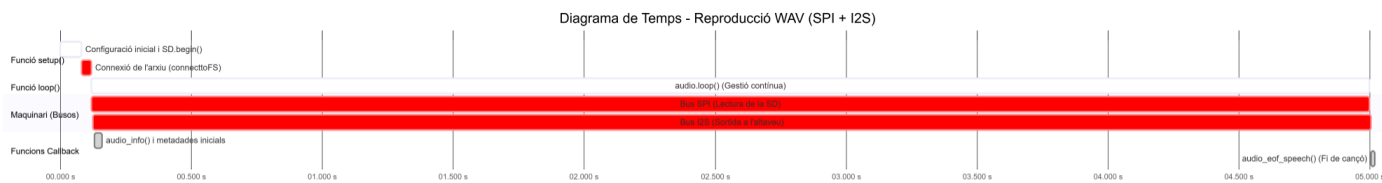


Diagrama de temps



Inici i Metadades (120 - 160 ms): Just després d'obrir l'arxiu i entrar al bucle `audio.loop()`, el programa llegeix les primeres dades (la capçalera de l'arxiu WAV). En aquest instant exacte, es disparen els *callbacks* asíncrons (franges grises) per informar-nos pel terminal sèrie del format, els canals i el *bitrate* de la cançó, sense aturar el processament d'àudio.

Paral·lelisme de Busos (120 - 5000 ms): Aquesta és la característica més important del diagrama. S'observen dues franges vermelles paral·leles i contínues que representen la saturació del maquinari. El bus SPI està constantment transferint dades de la targeta SD cap a la memòria de l'ESP32, mentre que, amb un decalatge d'uns pocs mil·lisegons, el bus I2S envia aquestes mateixes dades cap a l'altaveu. Tota aquesta gestió es manté viva gràcies a l'execució ininterrompuda de la funció `audio.loop()`.

Fi de l'Arxiu (5000 ms endavant): Un cop s'ha llegit l'últim byte per l'SPI i s'ha reproduït per l'I2S, els busos queden alliberats de la seva càrrega i es dispara un darrer esdeveniment automàtic (`audio_eof_speech()`) per avisar-nos de la fi de la reproducció.

Conclusions

La realització d'aquesta setena i última pràctica ha suposat un excel·lent tancament a l'estudi dels busos de comunicació. L'exploració del protocol **I2S** ens ha permès endinsar-nos en el processament de senyals multimèdia en temps real, un àmbit on el microcontrolador es posa al límit de les seves capacitats.

De l'evolució dels dos exercicis desenvolupats, se n'extreuen les següents conclusions fonamentals:

- **La necessitat de l'I2S per a l'Àudio Professional:** S'ha comprovat empíricament per què els DAC interns dels microcontroladors (habitualment de 8 bits) són insuficients per a la reproducció musical. L'ús del bus I2S juntament amb un descodificador extern com el MAX98357A permet transferir l'àudio de 16 bits de forma totalment digital i sense pèrdues, aconseguint una qualitat de so impensable en plaques més bàsiques.
- **L'estrès Computacional (Descodificació vs. I/O):** La pràctica ha posat de manifest dues maneres d'estressar un microcontrolador. L'exercici 1 ens ha ensenyat que descodificar un arxiu comprimit (AAC) des de la memòria flaix interna (PROGMEM) requereix una potència matemàtica sostinguda molt alta. En canvi, l'exercici 2 ens ha mostrat el concepte de "pont de dades", on la CPU no calcula, sinó que es dedica frenèticament a moure blocs d'informació d'un bus (SPI) cap a un altre (I2S).

- **Convivència i Paral·lelisme de Busos:** El segon muntatge és, sens dubte, un dels més complexos a nivell de maquinari. S'ha demostrat que l'ESP32 és capaç de gestionar un bus SPI funcionant a 4 MHz per llegir la targeta SD i, simultàniament, mantenir un flux de dades síncron i perfectament temporitzat pel bus I2S cap a l'altaveu, tot això sense que es produeixin talls ni interrupcions en la cançó.
- **Esdeveniments Asíncrons (Callbacks):** L'ús de llibreries avançades com `ESP32-audioI2S` ens ha introduït al paradigma de la programació asíncrona. Hem vist com el sistema és capaç d'informar-nos en temps real sobre metadades (bitrate, canals, fi de l'arxiu) en segon pla, sense arribar a bloquejar mai el bucle principal de reproducció de la cançó.

En definitiva, aquesta pràctica ens ha proporcionat una visió molt realista sobre com es dissenyen els reproductors d'àudio integrats o els altaveus intel·ligents d'avui en dia. Unint l'I2C, l'SPI i l'I2S, ara disposem d'una base tècnica robusta per afrontar pràcticament qualsevol repte d'integració de sensors, memòries i perifèrics multimèdia en projectes d'Internet de les Coses (IoT).